

JavaScript Strings

| What is a String?

DEFINITION

A **String** is a sequence of characters used to represent text in JavaScript.

A string can contain:

- Letters
- Numbers
- Symbols
- Spaces
- Special Characters

Strings are enclosed in:

- Single Quotes ()
- Double Quotes ()
- Backticks ()

| Why Do We Use Strings?

Strings are used to store textual data such as:

- User Name
- Email
- Password
- Address
- Messages

- Product Name
 - City Name
-

Creating Strings

There are three ways to create a string.

1. USING DOUBLE QUOTES

```
let name = "Rokon";  
  
console.log(name);
```

OUTPUT

```
Rokon
```

2. USING SINGLE QUOTES

```
let country = 'Bangladesh';  
  
console.log(country);
```

OUTPUT

Bangladesh

3. USING BACKTICKS (TEMPLATE LITERALS)

```
let language = `JavaScript`;  
  
console.log(language);
```

OUTPUT

JavaScript

| String Examples

```
let firstName = "MD";  
let lastName = "Rokonuzzaman";  
  
console.log(firstName);  
console.log(lastName);
```

OUTPUT

```
MD
Rokonuzzaman
```

| String Length

DEFINITION

The **length** property returns the total number of characters in a string.

It counts:

- Letters
- Numbers
- Spaces
- Symbols

Everything counts as one character.

| Syntax

```
string.length
```

Example 1

```
let name = "JavaScript";  
  
console.log(name.length);
```

OUTPUT

```
10
```

Example 2

```
let text = "Hello World";  
  
console.log(text.length);
```

OUTPUT

```
11
```

Explanation

```
H e l l o _ W o r l d  
1 2 3 4 5 6 7 8 9 10 11
```

Space is also counted.

Real Project Example

Check password length.

```
let password = "abc12345";  
  
if(password.length >= 8){  
  console.log("Strong Password");  
}else{  
  console.log("Weak Password");  
}
```

OUTPUT

```
Strong Password
```

Character Access

Definition

Each character in a string has its own position called an **index**.

JavaScript starts counting from **0**.

Example

```
let language = "JavaScript";
```

Character

J a v a S c r i p t

Index

0 1 2 3 4 5 6 7 8 9

| Access Using Index

```
let language = "JavaScript";  
  
console.log(language[0]);  
console.log(language[1]);  
console.log(language[4]);
```

OUTPUT

```
J  
a  
s
```

| Last Character

```
let language = "JavaScript";  
  
console.log(language[language.length-1]);
```


OUTPUT

t

| Loop Through a String

```
let text = "Code";

for(let i=0;i<text.length;i++){

  console.log(text[i]);

}
```

OUTPUT

C
o
d
e

Immutable

Definition

Strings are **immutable**.

Immutable means **once a string is created, its characters cannot be changed directly**.

You cannot modify an existing character using its index.

Wrong Example

```
let name = "Rokon";  
  
name[0] = "M";  
  
console.log(name);
```

OUTPUT

```
Rokon
```

Nothing changes.

| Why?

JavaScript does not allow changing individual characters of a string.

Instead, create a new string.

| Correct Way

```
let name = "Rokon";  
  
name = "Mokon";  
  
console.log(name);
```

OUTPUT

```
Mokon
```

Another Example

```
let text = "Hello";  
  
text = text + " World";  
  
console.log(text);
```

OUTPUT

```
Hello World
```

JavaScript creates a **new string** instead of changing the old one.

Escape Characters

Definition

Escape characters allow special characters to be included inside a string.

They start with a backslash (`\`).

Common Escape Characters

Escape	Meaning
<code>\"</code>	Double Quote
<code>\'</code>	Single Quote
<code>\\</code>	Backslash
<code>\n</code>	New Line
<code>\t</code>	Tab

Double Quote Example

```
let text = "He said, \"Hello\"";  
  
console.log(text);
```

OUTPUT

```
He said, "Hello"
```

| Single Quote Example

```
let text = 'It\'s JavaScript';  
  
console.log(text);
```

OUTPUT

```
It's JavaScript
```

| New Line

```
let text = "HTML\nCSS\nJavaScript";  
  
console.log(text);
```

OUTPUT

```
HTML  
CSS  
JavaScript
```

| Tab

```
let text = "Name\tAge";  
  
console.log(text);
```

OUTPUT

```
Name    Age
```

| Backslash

```
let path = "C:\\Users\\Admin";  
  
console.log(path);
```

OUTPUT

```
C:\Users\Admin
```

Best Practices

- ✓ Use double quotes or single quotes consistently.

```
let name = "Rokon";
```

- ✓ Use template literals when inserting variables.

```
let name = "Rokon";  
  
console.log(`Welcome ${name}`);
```

- ✓ Use `.length` instead of manually counting characters.
-

Common Mistakes

FORGETTING QUOTES



```
let name = Rokon;
```


Error

WRONG INDEX

```
let name = "Java";  
  
console.log(name[10]);
```

Output

```
undefined
```

TRYING TO MODIFY A CHARACTER



```
let name = "Java";  
  
name[0] = "C";
```

Output

```
Java
```

Interview Questions

| What is a String?

A string is a sequence of characters used to represent text in JavaScript.

| What is String Length?

The `length` property returns the total number of characters in a string.

| What does Immutable mean?

Immutable means the value cannot be changed after it is created. Strings in JavaScript are immutable.

| What is Character Index?

An index is the position of a character inside a string. Indexing starts from **0**.

Summary

Topic	Description
String	Sequence of characters
Quotes	Single, Double, Backticks
length	Returns total characters
Character Access	Uses index starting from 0
Immutable	Characters cannot be changed directly
Escape Characters	Special characters inside strings

JavaScript String Methods (Part 2)

This section covers the most commonly used string methods in JavaScript.

- `toLowerCase()`
 - `toUpperCase()`
 - `trim()`
 - `slice()`
 - `substring()`
 - `concat()`
 - `includes()`
 - `startsWith()`
 - `endsWith()`
-

1. toLowerCase()

Definition

The `toLowerCase()` method converts all characters in a string to lowercase.

It does **not** change the original string. It returns a new string.

Syntax

```
string.toLowerCase()
```

Example

```
let text = "HELLO WORLD";  
  
console.log(text.toLowerCase());
```

OUTPUT

```
hello world
```

| Real Project Example

Email comparison.

```
let email = "Admin@Gmail.com";  
  
console.log(email.toLowerCase());
```

Output

```
admin@gmail.com
```

2. toUpperCase()

| Definition

The `toUpperCase()` method converts all characters to uppercase.

It returns a new string.

Syntax

```
string.toUpperCase()
```

Example

```
let text = "javascript";  
  
console.log(text.toUpperCase());
```

OUTPUT

```
JAVASCRIPT
```

Real Project Example

```
let country = "bangladesh";  
  
console.log(country.toUpperCase());
```

Output

```
BANGLADESH
```

3. trim()

| Definition

The `trim()` method removes whitespace from the beginning and the end of a string.

It does not remove spaces between words.

| Syntax

```
string.trim()
```

Example

```
let text = "  Hello World  ";  
  
console.log(text.trim());
```

OUTPUT

```
Hello World
```

Without trim()

```
"  Hello World  "
```

After trim()

```
"Hello World"
```

| Real Project Example

User input.

```
let username = "  Rokon  ";  
  
console.log(username.trim());
```

Output

Rokon

4. slice()

| Definition

The `slice()` method extracts a part of a string.

The original string is not changed.

Syntax

```
string.slice(start, end)
```

- Start index is included.
 - End index is excluded.
-

Example

```
let language = "JavaScript";  
  
console.log(language.slice(0,4));
```

OUTPUT

```
Java
```

Example 2

```
console.log(language.slice(4));
```

Output

Script

Negative Index

```
let text = "JavaScript";  
  
console.log(text.slice(-6));
```

Output

Script

Real Project Example

```
let phone = "01712345678";  
  
console.log(phone.slice(-4));
```

Output

```
5678
```

5. substring()

Definition

The `substring()` method extracts characters from a string.

Unlike `slice()`, it does not accept negative indexes.

Syntax

```
string.substring(start,end)
```

Example

```
let language = "JavaScript";  
  
console.log(language.substring(0,4));
```

OUTPUT

```
Java
```

Example

```
console.log(language.substring(4));
```

Output

```
Script
```

| Difference Between slice() and substring()

slice()	substring()
Supports negative index	Does not support negative index
More commonly used	Less commonly used

6. concat()

| Definition

The `concat()` method joins two or more strings.

| Syntax

```
string.concat(string2)
```

Example

```
let firstName = "MD";  
let lastName = "Rokon";  
  
console.log(firstName.concat(" ",lastName));
```

OUTPUT

```
MD Rokon
```

Another Way

```
console.log(firstName + " " + lastName);
```

Modern JavaScript usually uses:

```
`${firstName} ${lastName}`
```

7. includes()

Definition

The `includes()` method checks whether a string contains a specific value.

It returns:

- true
 - false
-

Syntax

```
string.includes(searchValue)
```

Example

```
let language = "JavaScript";  
  
console.log(language.includes("Script"));
```


OUTPUT

```
true
```

| Example

```
console.log(language.includes("Python"));
```

Output

```
false
```

| Real Project Example

```
let email = "admin@gmail.com";  
  
console.log(email.includes("@"));
```

Output

```
true
```

8. startsWith()

Definition

The `startsWith()` method checks whether a string starts with specific characters.

Returns:

- true
 - false
-

Syntax

```
string.startsWith(value)
```

Example

```
let language = "JavaScript";  
  
console.log(language.startsWith("Java"));
```

OUTPUT

```
true
```

Example

```
console.log(language.startsWith("Script"));
```

Output

```
false
```

| Real Project Example

Check URL.

```
let url = "https://google.com";  
  
console.log(url.startsWith("https"));
```

Output

```
true
```

9. endsWith()

| Definition

The `endsWith()` method checks whether a string ends with specific characters.

Returns:

- true
 - false
-

Syntax

```
string.endsWith(value)
```

Example

```
let language = "JavaScript";  
  
console.log(language.endsWith("Script"));
```

OUTPUT

```
true
```

Example

```
console.log(language.endsWith("Java"));
```

Output

```
false
```

Real Project Example

Check file extension.

```
let file = "resume.pdf";  
  
console.log(file.endsWith(".pdf"));
```

Output

```
true
```

Summary Table

Method	Purpose	Returns
toLowerCase()	Converts to lowercase	New String
toUpperCase()	Converts to uppercase	New String
trim()	Removes leading and trailing spaces	New String
slice()	Extracts part of a string	New String
substring()	Extracts part of a string	New String
concat()	Joins strings	New String
includes()	Checks if text exists	Boolean
startsWith()	Checks starting text	Boolean
endsWith()	Checks ending text	Boolean

Most Frequently Used Methods



- trim()
- toLowerCase()
- includes()
- slice()



- startsWith()
- endsWith()
- concat()



- substring()

Interview Questions

Q1. What is the difference between slice() and substring()?

slice()	substring()
Supports negative index	Does not support negative index
More flexible	Simpler

Q2. Which method removes extra spaces?

```
trim()
```

Q3. Which method checks whether a string contains another string?

```
includes()
```

Q4. Which method converts a string to lowercase?

```
toLowerCase()
```

Q5. Which method checks the beginning of a string?

```
startsWith()
```

Q6. Which method checks the ending of a string?

```
endsWith()
```